

Business Logic Vulnerability – Unauthorized Refund Manipulation crAPI :

Overview

During security testing of the intentionally vulnerable **crAPI** application, I identified a business logic vulnerability in the product return workflow.

A normal authenticated user was able to manipulate order-related parameters and receive a refund significantly greater than the amount originally paid.

By modifying the product quantity from **1 to 30** and changing the order status from Pending to Returned, the application processed a **\$300 refund** for a product that originally cost **\$10**.

Testing Environment: crAPI deliberately vulnerable API lab

Authorization: Educational and authorized lab testing only

Vulnerability Details

Vulnerability Type: Business Logic Vulnerability

Related Weakness: Improper Server-Side Validation / Unauthorized Modification of Sensitive Properties

Potential OWASP API Category: API3:2023 – Broken Object Property Level Authorization

Impact

An authenticated low-privileged user could manipulate the return workflow to receive an unauthorized refund.

In this test:

- Original account balance: **\$100**
- Product price: **\$10**
- Purchased quantity: **1**
- Balance after purchase: **\$90**
- Manipulated return quantity: **30**
- Calculated refund: **\$300**
- Final account balance: **\$390**

This resulted in an unauthorized financial gain of **\$300** beyond the legitimate \$10 refund.

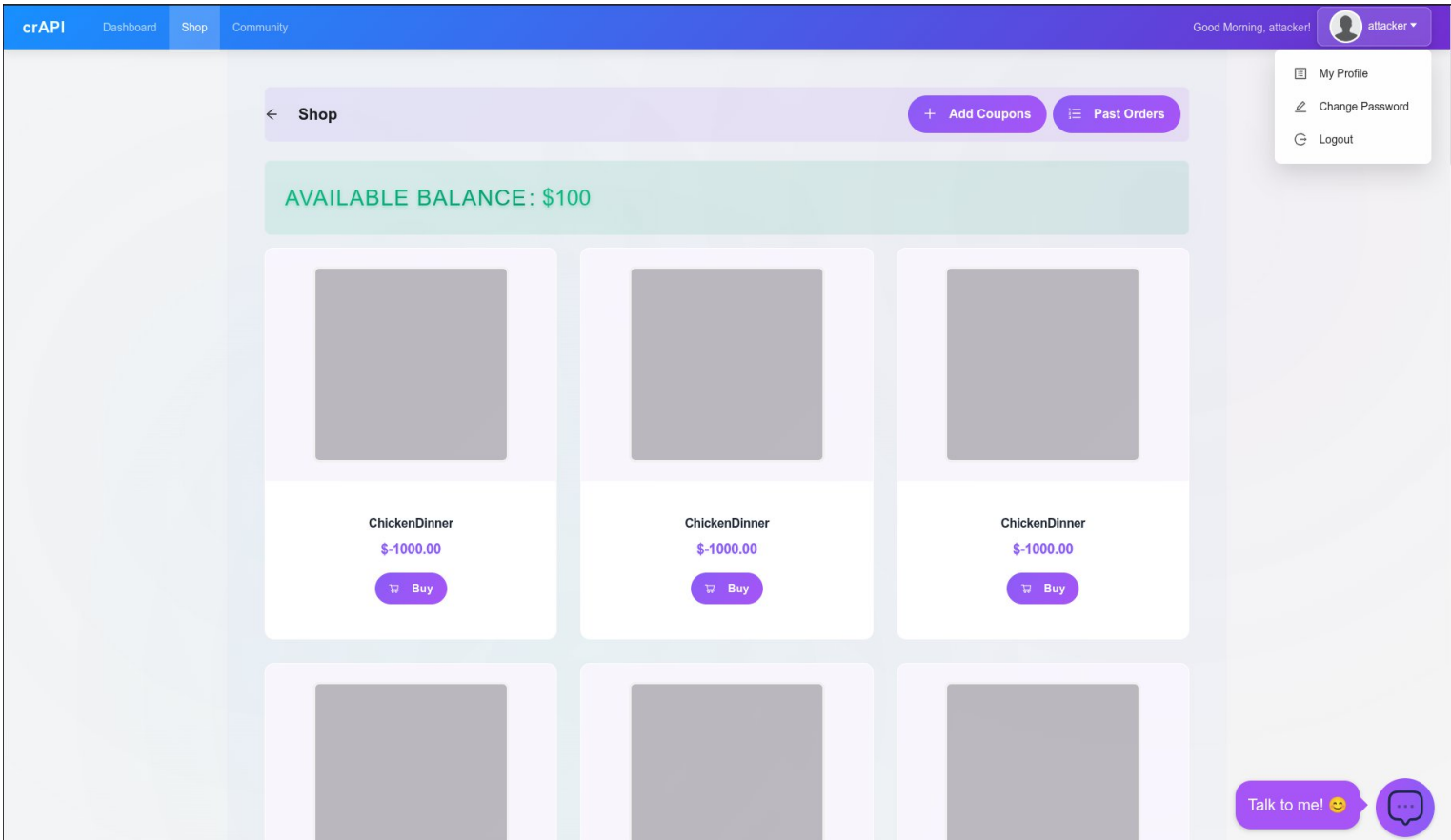
Attack Scnerio :

1) Normal User Account :

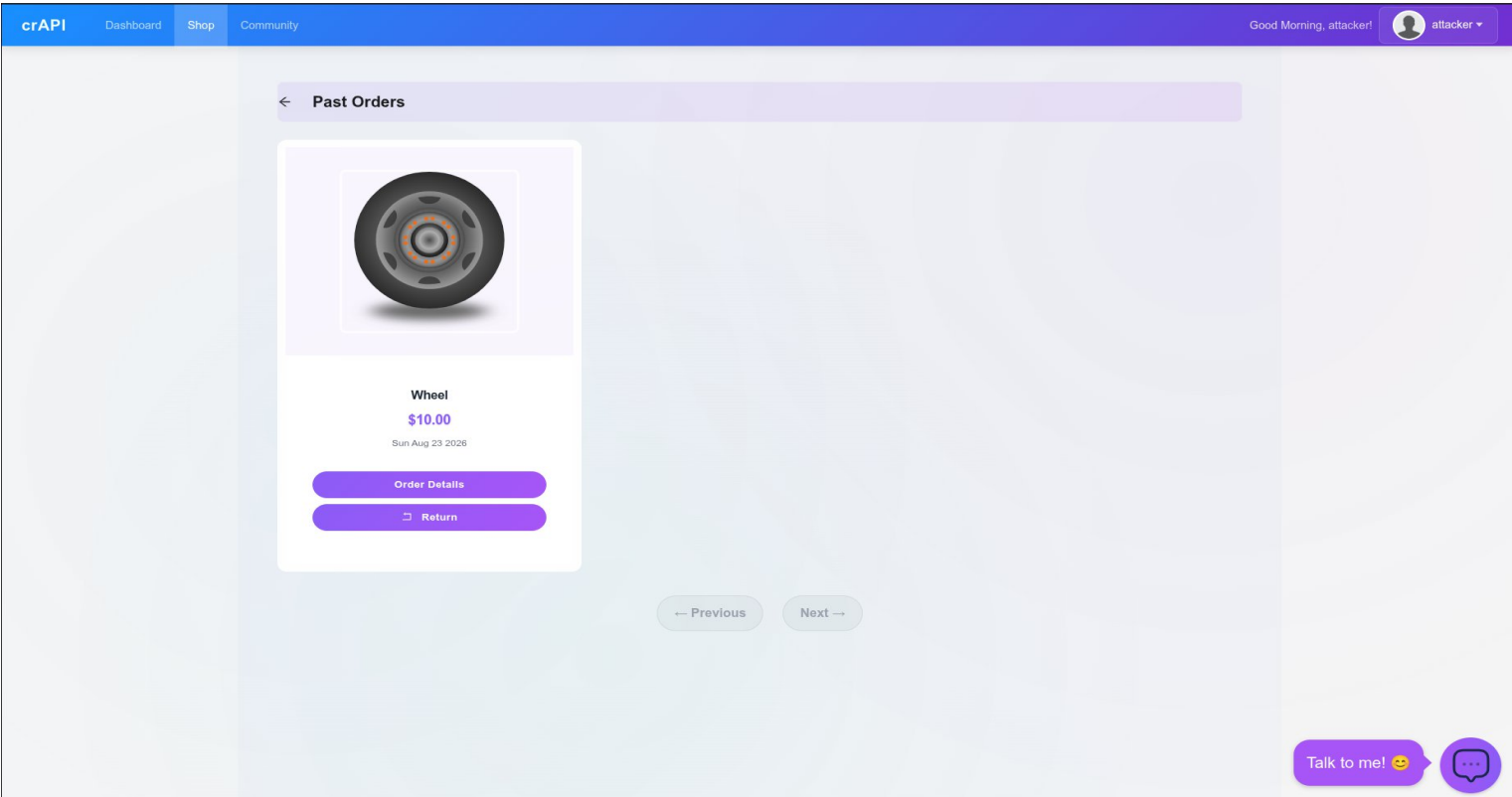
I registered and logged into crAPI as a normal authenticated user.

The account initially contained:

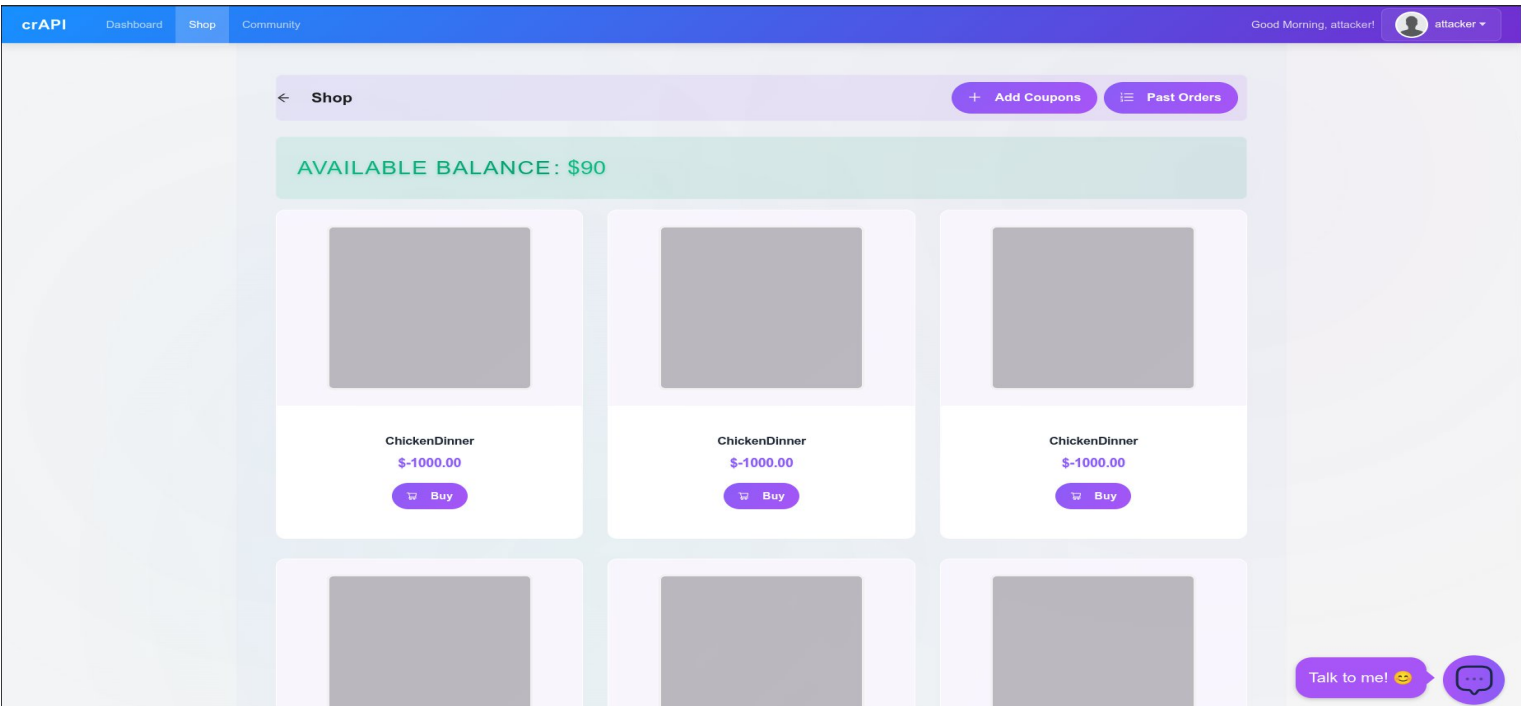
Available Balance: \$100



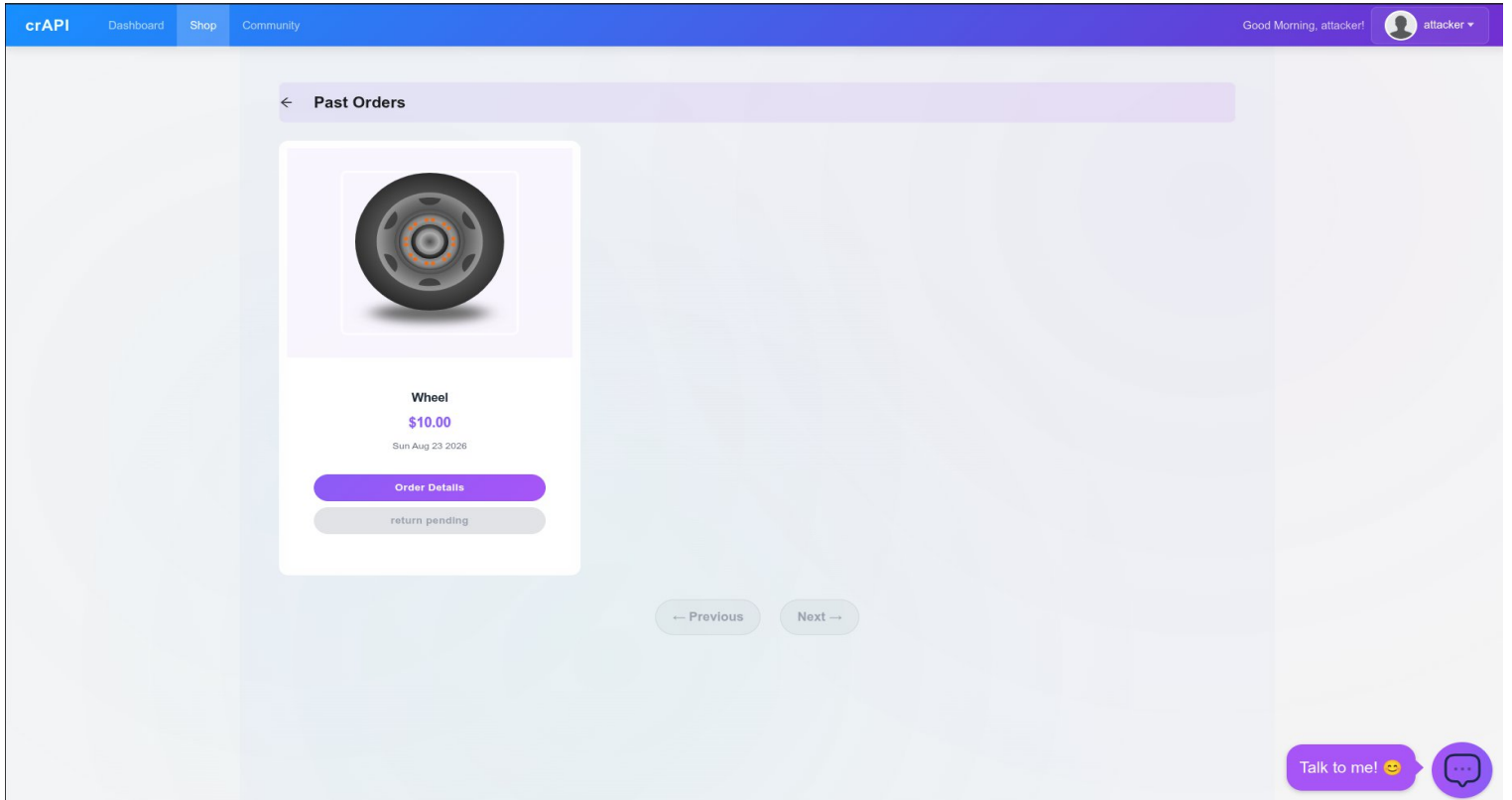
2) Purchase a product with a price \$10



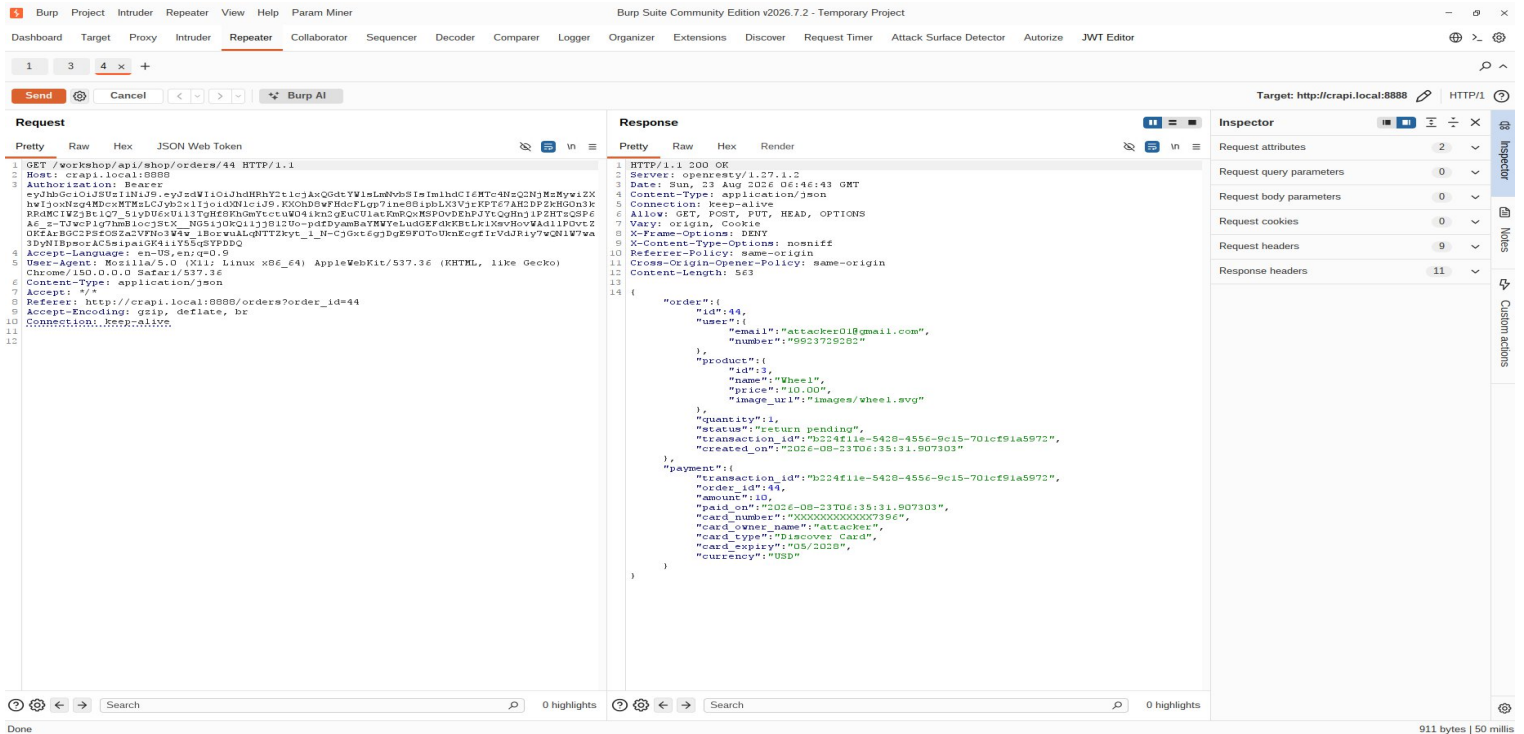
After purchase a product account balance is \$90



3) Initiate the return process



4) Captured Order Details request :



5) Manipulate sensitive properties

Burp Suite Community Edition - Temporarily Project

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscoverRequest TimerAttack Surface DetectorAuthorizeJWT Editor

SendCancel<>Burp AI

Request

PrettyRawHexJSON Web Token

```
GET /workshop/api/shop/orders/44 HTTP/1.1
Host: crapi.local:8888
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTQwIiwiaWF0IjoxODkxMTUyLjEKOABSWFdcFlap7lneSBBSjBMLDZVYtEPFTGTANCDFPKHGDm3ePRdMcWZ3BeIQ7_SlyPUExUil3TqHFSEKEmcttcutW04ikn2geUCULatKmPQokMSPovDEhPJTCgqhnlJPZHTsoCSP6Ae_-zuwPlgfThmbIooc3StX_MOSljOKclij3jl2Wo-pdrDYambaTHMYeludGEFdkEBELklxkvHovVadllPVoz2QRfAtNOCpfccfozaCVPMo3Wfw_HortualidGTT2kyr_i_M-CjGwtGsgjBgS9FOUenkeogfcitvdvbiywGNLUwa3dyNIIBporACCSipaiGkiiYS5gsYPDDO
Accept-Language: en-US,en;q=0.9
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Referer: http://crapi.local:8888/orders?order_id=44
Accept-Encoding: gzip, deflate, br
Content-Length: 897
Connection: keep-alive
```

Response

PrettyRawHexRender

```
HTTP/1.1 200 OK
Server: openresty/1.27.1.2
Date: Sun, 23 Aug 2026 06:46:43 GMT
Content-Type: application/json
Connection: keep-alive
Allow: GET, POST, PUT, HEAD, OPTIONS
Vary: origin, Cookie
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin
Content-Length: 563

{
  "orders": {
    "id": 44,
    "user": {
      "email": "attacker01@gmail.com",
      "number": "9923759202"
    },
    "product": {
      "id": 3,
      "name": "Wheel",
      "price": "10.00",
      "image_url": "images/wheel.svg"
    },
    "quantity": 1,
    "status": "return pending",
    "transaction_id": "bc224file-5428-4556-9c15-701cf91a5972",
    "created_on": "2026-08-23T06:35:31.907303"
  },
  "payment": {
    "transaction_id": "bc224file-5428-4556-9c15-701cf91a5972",
    "order_id": 44,
    "amount": 10,
    "paid_on": "2026-08-23T06:35:31.907303",
    "card_number": "XXXXXXXXXXXXXXX7396",
    "card_owner_name": "Attacker",
    "card_type": "Discover Card",
    "card_expiry": "N/S/2028",
    "currency": "USD"
  }
}
```

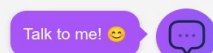
Inspector

Request attributesRequest query parametersRequest body parametersRequest cookiesRequest headersResponse headers

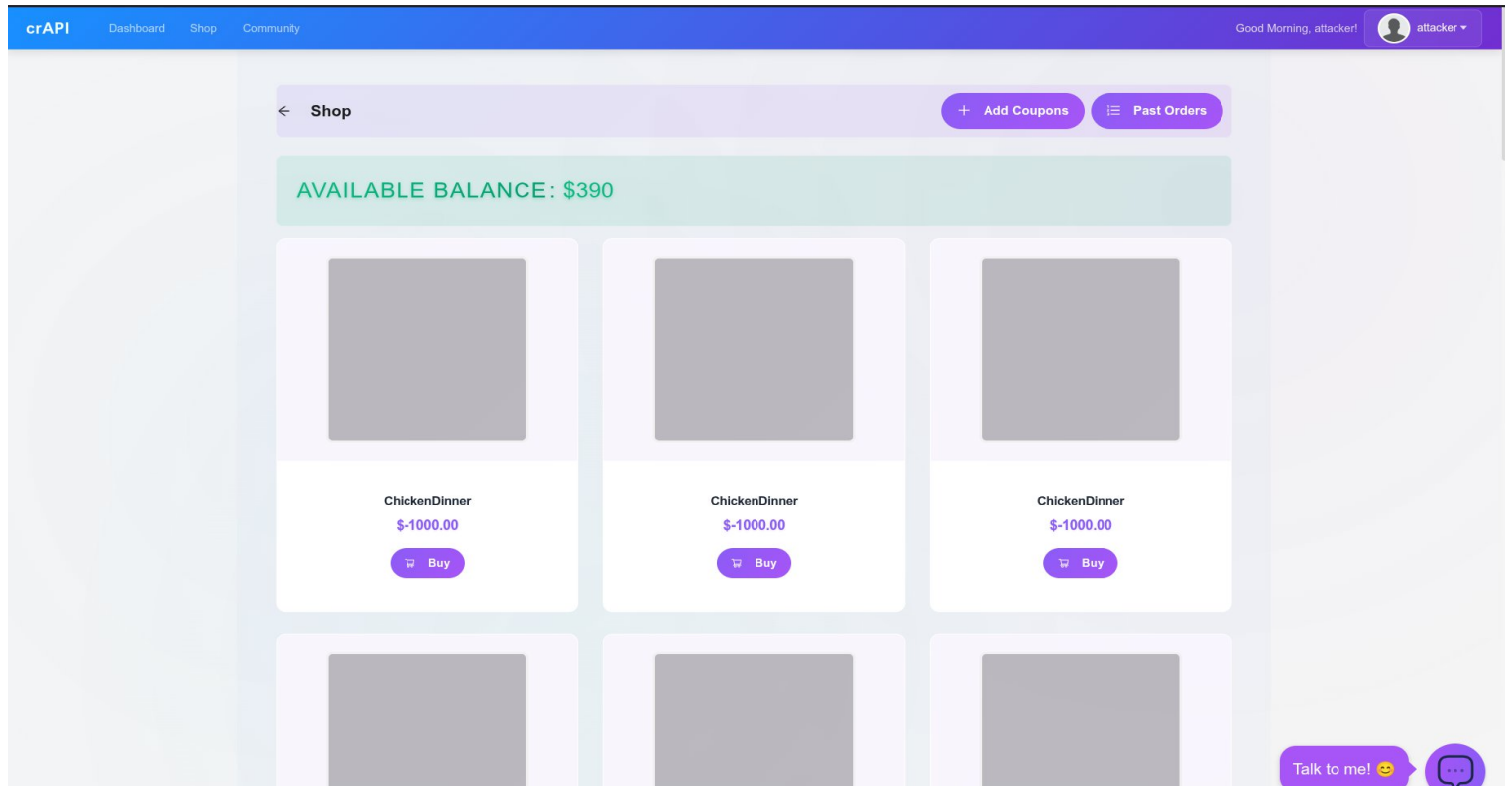
Done911 bytes | 50 milli

Change the Quantity 1 to 30

[illegible]



6) Unauthorized Refund :



The backend processed the manipulated quantity and calculated the refund based on **30 products** rather than the **1 product actually purchased**. The manipulated return was accepted, and the account received a **\$300 refund**.

Root Cause :

The vulnerability appears to occur because the server trusted client-controlled order properties during the return and refund workflow.

The backend should not rely on values such as:

- Product quantity
- Product price
- Refund amount
- Order status

when those values are supplied or modified by the client. Instead, the server should retrieve the original order information from a trusted server-side data source and validate the return against that information.

Security Impact :

A malicious authenticated user could potentially:

- Manipulate return quantities
- Obtain refunds exceeding the original purchase amount
- Increase their account balance without authorization
- Abuse weaknesses in financial business workflows

In a real-world e-commerce application, this type of flaw could lead to **direct financial loss**.

Remediation :

- I. Validate return quantities server-side
- II. Calculate refunds using trusted server-side data
- III. Protect sensitive order properties
- IV. Enforce valid state transitions
- V. Validate the complete transaction

Key Takeaway :

This finding demonstrates why security testing should not focus only on traditional vulnerabilities such as SQL injection or authentication bypasses.

Even when an API correctly authenticates a user, insecure business rules and insufficient server-side validation can allow an attacker to abuse legitimate functionality for unauthorized financial gain.

Tested only in the intentionally vulnerable crAPI lab environment for educational and authorized security research.